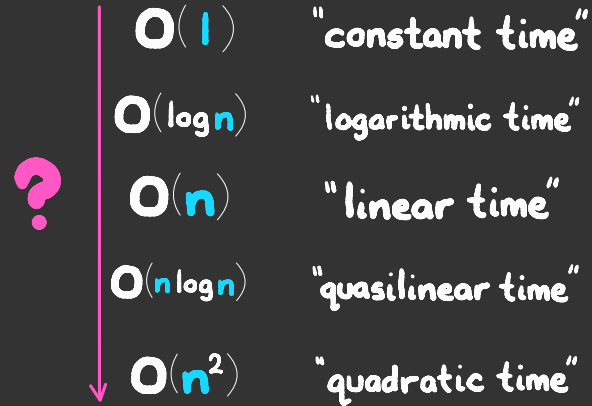


analyzing
code

CSCI
134

given a python function, how do we determine its complexity class?

```
def is_perfect(n):  
    total = 0  
    for i in range(1, n):  
        if n % i == 0:  
            total += i  
    return n == total
```



count the operations (worst-case scenario)

| assignment

| condition check
| arithmetic op (%)

| increment

| condition check

```
def is_perfect(n):  
    total = 0  
    for i in range(1, n):  
        if n % i == 0:  
            total += i  
    return n == total
```

count the **operations**
(worst-case scenario)

$n-1$
iterations

```
def is_perfect(n):
```

| assignment

```
    for i in range(1, n):
```

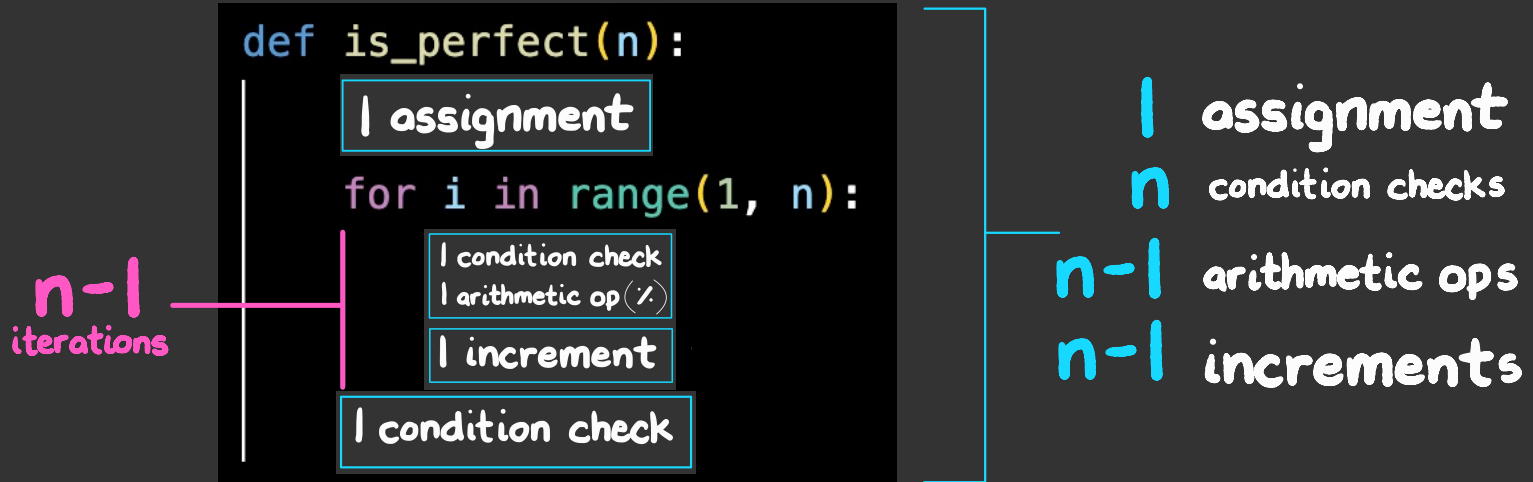
| condition check

| arithmetic op (%)

| increment

| condition check

count the operations (worst-case scenario)



determine the complexity of each operation

```
def is_perfect(n):
```

```
    | assignment
```

```
    for i in range(1, n):
```

```
        | condition check
```

```
        | arithmetic op (x/)
```

```
        | increment
```

```
    | condition check
```

1 assignment $O(1)$

n condition checks $O(1)$

n-1 arithmetic ops $O(1)$

n-1 increments $O(1)$

all
constant time

do the math

```
def is_perfect(n):  
    | assignment  
    for i in range(1, n):  
        | condition check  
        | arithmetic op (/)  
        | increment  
    | condition check
```

$$\begin{array}{r} 1 \\ n \\ n-1 \\ + n-1 \end{array} \quad \begin{array}{c} * \\ * \\ * \\ * \end{array} \quad \begin{array}{c} | \\ | \\ | \\ | \end{array}$$

$$3n-2$$

determine the complexity class

```
def is_perfect(n):
```

```
    | assignment
```

```
    for i in range(1, n):
```

```
        | condition check
```

```
        | arithmetic op (✓)
```

```
        | increment
```

```
    | condition check
```

$$3n - 2 \in O(n)$$

given a python function, how do we determine its complexity class?

```
def is_perfect(n):  
    total = 0  
    for i in range(1, n):  
        if n % i == 0:  
            total += i  
    return n == total
```

$O(1)$ "constant time"

$O(\log n)$ "logarithmic time"

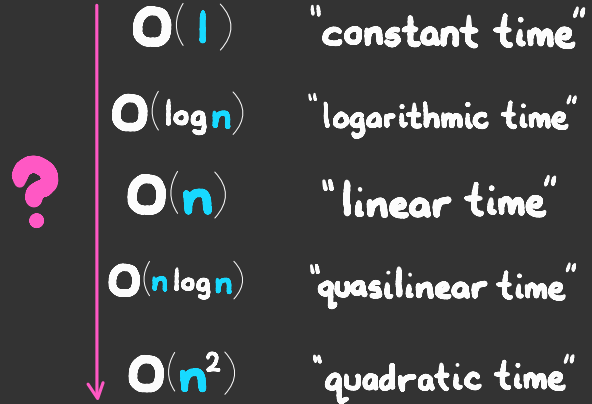
✓ $O(n)$ "linear time"

$O(n \log n)$ "quasilinear time"

$O(n^2)$ "quadratic time"

let's try another

```
def perfect_numbers_up_to(n):  
    result = []  
    for i in range(1, n+1):  
        if is_perfect(i):  
            result.append(i)  
    return result
```



$O(1)$	"constant time"
$O(\log n)$	"logarithmic time"
$O(n)$	"linear time"
$O(n \log n)$	"quasilinear time"
$O(n^2)$	"quadratic time"

count the **operations** (worst-case scenario)

| assignment

| **is_perfect** call

| list append

```
def perfect_numbers_up_to(n):  
    result = []  
    for i in range(1, n+1):  
        if is_perfect(i):  
            result.append(i)  
    return result
```

count the **operations**
(worst-case scenario)

n
iterations

```
def perfect_numbers_up_to(n):
```

| assignment

```
    for i in range(1, n+1):
```

| is_perfect call

| list append

```
    return result
```


count the operations (worst-case scenario)

n
iterations

```
def perfect_numbers_up_to(n):
```

| assignment

```
    for i in range(1, n+1):
```

| is_perfect call

| list append

```
    return result
```

| assignment

n is_perfect calls

n list appends

determine the complexity of each operation

```
def perfect_numbers_up_to(n):  
    | assignment  
    for i in range(1, n+1):  
        | is_perfect call  
        | list append  
    return result
```

1 assignment $O(1)$

n is_perfect calls $O(n)$

n list appends $O(1)$

do the math

```
def perfect_numbers_up_to(n):  
    | assignment  
    for i in range(1, n+1):  
        | is_perfect call  
        | list append  
    return result
```

$$\begin{array}{r} 1 \\ n \\ + n \\ \hline n^2 + n + 1 \end{array} \begin{array}{c} * \\ * \\ * \end{array} \begin{array}{r} 1 \\ n \\ 1 \end{array}$$

determine the complexity class

```
def perfect_numbers_up_to(n):  
    | assignment  
    for i in range(1, n+1):  
        | is_perfect call  
        | list append  
    return result
```

$$n^2 + n + 1 \in O(n^2)$$

remember how slow this was?

```
def perfect_numbers_up_to(n):  
    result = []  
    for i in range(1, n+1):  
        if is_perfect(i):  
            result.append(i)  
    return result
```

$O(1)$ "constant time"

$O(\log n)$ "logarithmic time"

$O(n)$ "linear time"

$O(n \log n)$ "quasilinear time"



$O(n^2)$ "quadratic time"